

Technical Architecture Analysis Report

Architecture Score: 4/10

Components Identified

- Bank Core Service (gestão do banco e ATMs)
- Customer Service (gestão de clientes, endereços)
- Account Service (gestão de contas, saldos e tipos)
- Card Service (gestão de cartões de débito e acesso)
- Transaction Service (gestão de transações ATM, incluindo saques e transferências)
- ATM Management Service (gestão da localização e identificação de transações por ATM)

Architectural Risks

- Conflitos de Consistência de Dados Distribuídos: A manipulação de saldos em 'Account' por múltiplas 'ATMTransaction' em um ambiente distribuído é um desafio significativo sem estratégias claras (e.g., Sagas, Event Sourcing).
- Falta de Mecanismos de Segurança Explícitos: O diagrama não aborda autenticação, autorização, criptografia ou proteção contra fraudes, que são críticas para um sistema bancário.
- Acoplamento Potencialmente Alto Entre Serviços: A forte interconexão entre as classes (ex: 'Customer' possui 'Account' e 'DebitCard', 'Account' é modificada por 'ATMTransaction') pode levar a um acoplamento indesejado se traduzido diretamente para microsserviços.
- Pontos de Falha Únicos e Gargalos de Escalabilidade: A centralização aparente de 'ATMTransaction' e 'Account' como entidades-chave pode criar gargalos de desempenho e pontos de falha em um sistema de alta carga.
- Ausência de Práticas de Observabilidade: Não há indicação de como o sistema será monitorado, logado ou rastreado, o que é fundamental para depuração e operação em sistemas distribuídos.

Recommendations

- Decomposição com Bounded Contexts: Refinar a arquitetura de microsserviços usando Domain-Driven Design (DDD) para definir limites claros de contexto, como 'Customer Context', 'Account Context', 'Card Context', 'Transaction Context'.
- Adoção de Arquitetura Orientada a Eventos: Implementar um message broker (e.g., Kafka) para comunicação assíncrona entre microsserviços, usando eventos para propagar mudanças de estado e Sagas para orquestração de transações distribuídas (e.g., débito/crédito).
- Implementação de uma Camada de Segurança Robusta: Introduzir um API Gateway com autenticação (OAuth2/OpenID Connect), autorização granular, validação de requisições e criptografia (TLS) para todas as comunicações externas e internas.
- Estratégias de Tolerância a Falhas e Escalabilidade: Aplicar padrões de resiliência como Circuit Breakers, Retries e Bulkheads. Considerar sharding de dados para 'Account' e 'ATMTransaction' para suportar alta carga e distribuir o armazenamento.
- Estabelecimento de Plataforma de Observabilidade: Implementar uma solução completa de logs centralizados (ELK/Grafana Loki), monitoramento de métricas (Prometheus/Grafana) e

rastreamento distribuído (Jaeger/OpenTelemetry) para visibilidade operacional.